Kafka 消费积压应急处置手册

1 文档信息

本文档编号 OPS-KAFKA-020，版本 V2.1，密级内部公开，由基础架构部消息队列运维组编写并维护，适用于公司所有使用 Kafka 消息队列的业务系统在发生消费积压时的应急处置，覆盖生产环境与预发布环境，测试环境可参照执行但不强制要求留存记录。本文档编写人为消息队列运维组陈晨，审核人为基础架构部技术委员会与消费平台架构组联审，批准人为基础架构部负责人。本版本发布于 2026 年 3 月，自发布之日起 V2.0 版本自动作废，值班室张贴的处置流程图需同步更换为新版。手册内容与线上实际行为不一致时，以先止血后追因为原则，先按应急章节处置，事后再修正手册，不允许在现场与手册条文反复纠缠而延误恢复时限。

修订记录说明如下。V1.0 版本只包含积压度量的基本方法与扩容消费者的步骤；V1.5 版本补充了消费组再均衡风暴的诊断章节，引入心跳会话超时与拉取间隔两类参数的区分方法；V2.0 版本把消费提速的三个层次固化为处置顺序，并增加紧急场景的快速消化预案；V2.1 版本补充了消费者编号列定位假死实例的方法，并把告警基线从经验值修订为按业务分级设定。历次修订的评审纪要存于运维知识库，编号 OPS-KAFKA-020-CHANGELOG，追溯旧版按编号调阅。

2 概述

消费积压是消息队列最常见的服务劣化形态，表现是消息生产速度持续高于消费速度，未处理消息在分区里不断堆积。积压本身不是故障，而是消费能力与生产能力失衡的信号，处置的核心思路是先判断形态再动手，不同的形态对应完全不同的处置路径，路径选错的代价是浪费处置时间甚至把问题放大。本手册覆盖积压的发现与度量、消费组再均衡问题的诊断、消费端线程与批量参数的调优、扩容消费者的操作规程、紧急场景的快速消化预案五个部分。手册不涉及 Broker 侧的运维操作与 ISR 副本同步机制的原理分析，这两块内容由兄弟文档 OPS-KAFKA-030 承载，值班时遇到 Broker 进程异常或副本持续脱离同步的场景请直接转交该手册的处置流程，不要在本手册的路径里兜圈子。

消费积压的处置时效要求按业务分级。实时性业务（风控决策、支付回调）的积压处置时限为十分钟内启动、一小时内收敛；准实时业务（用户行为分析、搜索索引更新）的时限为三十分钟内启动、四小时内收敛；离线补数类业务的积压允许跨天处理，但要保证在数据加工链路的截止时间前消化完毕。分级归属清单由各业务方在接入时申报，经消费平台架构组确认后登记在消息平台的元数据里，值班同学处置积压告警前先查分级，再决定投入的力度与拉人的范围。分级登记不清晰的业务默认按准实时处置，事后由架构组与业务方补充确认。

3 积压的发现与度量

积压的度量指标是 Lag，定义为分区日志末端位移与消费组已提交位移的差值，Lag 乘以平均消息大小可以粗略折算出堆积的数据量。日常发现积压的渠道有两个：监控平台按消费组维度配置的 Lag 告警，以及值班巡检时的主动巡扫。手工核实的标准命令是 kafka-consumer-groups.sh 加 --describe 与 --group 参数，指定 bootstrap-server 与消费组名执行，输出按分区逐行列出 GROUP、TOPIC、PARTITION、CURRENT-OFFSET、LOG-END-OFFSET、LAG、CONSUMER-ID、HOST、CLIENT-ID 九列。看这份输出时重点不是单个数字，而是 Lag 在分区维度的分布形态，形态判断是后续所有处置动作的分岔点，看错形态后面全部白做。

形态一是个别分区 Lag 巨大而其他分区接近于零。这种形态大概率不是消费能力不足，而是该分区的消费宿主出了问题：进程还在但已经假死，心跳维持着但不再拉取消息，或者该分区的消费者实例正在反复加入退出。定位方法是看 CONSUMER-ID 列，正常情况下每个分区都有一个明确的消费者编号标识当前消费宿主，如果个别分区的消费者编号列显示为空缺或者与其它分区不一致，说明该分区处于无主或者刚经历再分配的状态；如果消费者编号都在，就顺着该行的 HOST 列找到宿主机器，登机器看进程的线程栈与 GC 日志，确认是否假死。假死实例的标准处置是先摘流量再重启，直接 kill 也能恢复，但要预期重启会触发一轮再均衡。

形态二是所有分区均匀堆积且持续增长。这种形态才是整体消费能力不足，进入扩容提速的处置路径，具体见本手册第五章与第六章。判断时还要叠加生产速率曲线：生产速率突增导致的均匀堆积，先确认突增是否为业务预期内的活动（大促、批量任务），预期内的突增且消费能力有冗余的，观察即可；生产速率平稳但消费速率下滑的均匀堆积，说明消费端在退化，退化往往与再均衡、下游依赖变慢、实例资源不足有关，先按第四章排查分区归属的重新分配因素，再查下游依赖，最后才考虑加机器。两种形态的处置手段完全不同，混着来会把简单问题复杂化，这句话在 V1.5 版本修订时被特别加粗强调过，再强调一次。

度量环节还有一个容易被忽视的细节是 Lag 的口径问题。kafka-consumer-groups.sh 加 --describe 参数看到的 Lag 是已提交位移口径，消费端拉取了但还没提交的部分不会体现在这个数字里，所以偶尔会出现 Lag 显示为零但业务还在堆积处理的现象，这是口径差异不是监控坏了。判断真实的处理进度要以业务侧的处理位点为准，或者在消息体里携带处理时间戳做端到端的对账。值班同学在汇报积压规模时要注明口径，两种口径的数字混在一张表里汇报，会误导上层对处置进度的判断，2025 年的大促保障期间就出现过口径混用导致重复拉起两拨人处置同一个已收敛积压的乌龙，教训写进了本手册。

4 重平衡问题的诊断与治理

消费组重平衡是分区所有权在消费者实例间的再分配过程，再均衡期间整组停止消费，Lag 在每次再均衡期间加速增长。频繁再均衡在日志里的表现为周期性的分区再分配记录，间隔从几十秒到几分钟不等，伴随业务侧的消费停顿。治理前必须先区分触发原因，两类超时参数对应两类完全不同的根因，参数调反了是把再均衡变成慢性病的最快方式。第一类是 session.timeout.ms 超时，该参数定义心跳会话的超时时间，消费端处理消息太慢导致心跳线程挤不出 CPU，协调者误判实例死亡而触发再均衡。这类根因的特征是分区归属的重新分配发生时实例其实还活着，日志里能看到实例被动的再平衡记录，处理方向是把心跳会话超时参数适度调大（默认十秒，可放宽到十五到三十秒），同时检查实例的 GC 停顿与线程饥饿，心跳线程被业务线程压制的问题要从线程池隔离下手根治。

第二类是 max.poll.interval.ms 超时，该参数定义两次 poll 调用的最大允许间隔，消费一批消息的处理时间超过了这个间隔，消费者会主动离组，随后触发再均衡。这类根因的特征是日志里明确写着实例因 poll 间隔超时而离开消费组，处理方向有两条：调大拉取间隔上限给业务处理留足时间，或者把单批拉取条数调小让每批处理量变小从而缩短处理周期。两条方向的选择依据是业务对延迟的敏感度，延迟敏感的业务优先减小批量，不敏感的业务优先调大间隔。需要特别提醒的是单批拉取条数与拉取间隔上限是一对联动参数，把批量调大的同时不调大间隔，等于亲手制造下一轮超时，这样的误操作在 2025 年的第二季度发生过三起，全都演变成了持续半天的再均衡风暴

。

两类参数的调整都要配合日志确认闭环。消费端日志里会写明离组原因是 session 超时还是 poll 间隔超时，按日志对症下药，调整后观察一整个业务周期（至少覆盖一次高峰）没有再出现再均衡，才算治理完成。调整消费组参数的操作路径是在发布平台修改消费者配置并滚动重启实例，重启本身会触发一次正常的消费者重新分组，属于预期行为，不要把这次预期内的再均衡误判为治理失败。重平衡治理的验收标准是二十四小时内非计划再均衡次数为零，计划内发布引起的除外。频繁再均衡与积压互为因果，再均衡停不下来积压就不可能收敛，所以分区归属的重新分配因素的排查在处置顺序上排在一切提速手段之前。

5 消费者线程与批量参数的调优

确认是整体能力不足且再均衡因素已排除后，提速按三个层次依次考虑，顺序不能乱，因为前一层的成本与风险都低于后一层。第一层是提高单实例的消费并发度，核心参数是 consumeThreadMin 与 consumeThreadMax，这两个参数控制消费线程池的最小与最大线程数，把消费线程数上限从默认二十调大到五十，单实例的并行消费能力立即提升，代价是实例的 CPU 与内存占用上升。调线程数之前必须核对下游资源：消费线程多了，下游数据库连接池、下游接口的配额都要同步扩，否则线程全部阻塞在等连接上，吞吐不升反降。实际踩过的坑是某业务把消费线程数上限调到八十，而下游连接池只有二十个连接，结果六十个线程常年排队，消费延迟反而恶化了百分之四十，回退后改分批调整才恢复正常。

第二层是批量消费，核心参数是 max.poll.records，控制单次 poll 拉回的最大消息数，默认五百。把单批拉取条数调大到两千甚至五千，配合业务侧把逐条处理改成批量提交，对数据库型下游的收益往往最大，一次批量写入摊薄了网络与事务开销，吞吐的提升经常是成倍的。调整 max.poll.records 有两个前提：一是处理逻辑必须改造为批量语义，逐条异常时不能把整批全部回滚，要有单条隔离的兜底；二是处理时长要重新核算，批量变大意味着单批处理时间变长，必须确保处理时间小于拉取间隔上限的设定，否则上一章讲过的 poll 超时离组就会找上门。批量消费的改造属于业务侧变更，需要业务方排期，处置现场用不上，但复盘时要作为长期优化项输出给业务方。

第三层才是水平扩容，增加消费者实例数。实例数受订阅主题的总分区数封顶，一个分区同一时刻只能被组内一个消费者消费，超过分区数的实例空转不接任何分区，纯粹浪费资源。扩容前先数清楚分区数，kafka-consumer-groups.sh 加 --describe 参数的输出里分区行数就是总分区数，或者直接查主题元数据。实例数对齐分区数就是并行度的数学上限，实例数等于分区数时并行度打满，之后想再提速只能回到第一二层或者扩分区，扩分区会破坏 key 的分区路由顺序性，必须业务方评估后才能操作，不在应急处置的权限范围内。

6 扩容消费者的操作规程

扩容消费者的标准操作分四步。第一步确认现状，记录当前实例数、分区数、每实例的消费线程数上限与单批拉取条数配置，折算出当前理论并行度，并行度没打满才谈得上扩容收益。第二步准备资源，扩容的实例规格必须与现有实例一致，启动参数与配置文件从发布平台的标准模板拉取，禁止手工在机器上改配置再启动，手工改出来的实例配置漂移是后续排查的噩梦。第三步滚动上线新实例，新实例注册进消费组会触发一次再均衡，分区所有权重新分配，这次再均衡是预期内的，业务侧会有几秒到几十秒的消费停顿，提前通知业务接口人。第四步验证，观察 Lag 曲线是否掉头向下、新实例是否分到了等量的分区、下游指标是否平稳，验证通过后扩容动作闭环。

扩容过程的注意事项有三条。第一条是分批上线，一次扩容超过现有实例数一半时，建议分两批间隔十分钟上线，避免一次再均衡波动的分区迁移面太大；第二条是错峰，扩容操作避开业务高峰，再均衡触发的消费停顿叠加高峰流量会放大业务感知；第三条是留退路，扩容后如果下游因消费提速而出现压力（数据库写入飙高、接口限流触发），要能快速缩回，缩容同样触发再均衡，评估时要把这层代价算进去。扩容完成后 Lag 的收敛速度可以折算成消化预计完成时间，公式是当前总 Lag 除以（消费速率减生产速率），结果超过业务容忍时限的，说明单靠扩容不够，转入紧急预案章节评估更强的手段。

7 紧急场景的快速消化

时效性已经丧失的极端积压有专门的预案，启用前提是业务方书面确认。千万级以上的积压且消息时效已过（比如活动结束后的行为日志、过期即作废的推荐刷新），继续消费只是浪费资源，最直接的手段是重置消费位点跳过积压段，把消费组的提交位移重置到当前最新位置，从新消息重新开始消费。重置操作的前提条件有三条：业务方在工单里书面确认跳过的消息不需要处理，这个确认必须留痕，口头确认一律无效；重置命令的执行人必须是指定的值班工程师，命令模板由消息平台封装好了操作入口，不开放原始命令防止误操作别的消费组；重置后半小时内持续观察消费是否正常追平生产。跳过的消息永久不处理，这条铁律要在确认环节向业务方反复强调。

时效性还在但常规提速来不及的场景，可以临时上线一批只做转发消费的实例，把这批实例的消费线程数上限拉到机器能承受的上限，单批拉取条数也同步放大，消费逻辑只有一个动作：把消息原样搬到下游能承受的缓冲介质里（临时主题、对象存储、本地磁盘队列），再由离线任务或者临时的批量消费者慢慢消化。这套方案的精髓是把不可控的实时消费压力转化为可控的离线处理，缓冲介质的容量要按积压数据量的一点五倍预留。任何紧急手段执行后都要保留现场：导出积压时段的消费组状态与分区位移快照、再均衡日志、当时段的消费与生产速率曲线，事后复盘积压产生的根因，是生产端突增还是消费端退化，复盘报告抄送业务方与架构组，避免同样的场景二次发生。

重置位点操作还有一个事后核对动作容易漏掉：跳过积压段之后，要再次执行 kafka-consumer-groups.sh 加 --describe 与 --group 参数，逐分区核对消费者编号与 CURRENT-OFFSET 是否已经推进到 LOG-END-OFFSET 附近，个别分区如果出现 CURRENT-OFFSET 停滞不动而消费者编号又显示为空，说明该分区没有跟上重置后的分配，需要按假死实例的路径单独处理。跳过段的原始数据建议在对象存储留一份备份，留档周期九十天，业务方事后想补算的部分可以从备份里捞出来离线补，这个兜底在两次真实事故里挽回过业务方的数据诉求，代价只是几十吉的存储成本，值得保留。

8 监控告警基线与日常巡检

积压的日常监控要做到分区粒度，监控数据同时保留消费速率与生产速率两条曲线，两条曲线的剪刀差是判断积压走向的最直观依据：消费速率贴着生产速率说明只是瞬时波动，不需要处置；消费速率长期低于生产速率说明结构性欠账，扩容是迟早的事，监控的价值在于让这个判断提前发生而不是等告警。告警基线按业务分级设定两级。一级预警：Lag 持续五分钟超过十万（实时业务）或者五十万（准实时业务），值班的处置动作是查看消费组状态与速率趋势，十五分钟内判断形态并回报；二级升级：Lag 持续三十分钟且速率没有收敛迹象，自动拉应急处置群，值班工程师按本手册流程处置，业务接口人同步到位。告警阈值的调整走监控平台配置变更，不允许值班同学现场随手改阈值了事，改阈值不能消灭积压。

日常巡检每周一次，巡检内容以段落形式说明。检查全量消费组的 Lag 排行榜，前十名之外的组要求零积压，前十名内的组逐个确认积压性质；抽查 kafka-consumer-groups.sh 加 --describe 参数的输出，核对消费者编号与 HOST 列，确认没有长期无主的分区；统计一周内非计划再均衡的次数，超过三次的消费组列入治理清单，按第四章的路径排查；核对各消费组的消费线程数上限与单批拉取条数配置与登记表一致，配置漂移的限期回正。巡检结果汇总为周报发布在消息队列运维群，治理清单跟踪到闭环为止。

9 附录

常用命令与参数速查以段落形式说明。查看消费组状态用 kafka-consumer-groups.sh 加 --describe 与 --group 参数；查看全部消费组列表用 kafka-consumer-groups.sh 加 --list 参数；两处命令都依赖 --bootstrap-server 指定接入点，接入点地址见消息平台的环境说明页。关键参数默认值速查：session.timeout.ms 默认十秒，建议按实例资源状况放宽到十五到三十秒；max.poll.interval.ms 默认五分钟，批量消费改造后必须重新核算；max.poll.records 默认五百，批量场景可放大到两千至五千；consumeThreadMax 默认二十，调大前先核对下游连接池与配额。参数的完整定义以消息平台的接入文档为准，本手册只收录与积压处置直接相关的部分。联系人与升级路径说明如下。消费平台架构组负责参数模板与消费组治理的技术支持，工作时间响应群消息两小时内；消息队列运维组值班表见值班系统，P2 级以上故障直接电话值班工程师；业务侧的接口人清单登记在消息平台的元数据里，按业务线分组，处置积压前先查分级与接口人再动手。本手册的修订建议提交给文档维护人，每个季度末集中评审，评审通过的修订随版本号发布，紧急的修订（例如新发现的踩坑案例）走快速通道当周发布。